Session 2: Conditions and Loops

Trainer Reference Guide

Purpose and outcomes

Translate business rules into if/elif/else branches and process repeated work with for, range, while, break, and continue.

Examples and exercises

- 01_conditionals.ipynb validates age/active state and guides department classification.
- 02_for_loops.ipynb numbers users and models bounded attempts. Guided work skips inactive records and stops at a target ID.
- 03_while_loops.py implements a three-attempt PIN workflow with clear remaining-attempt feedback.
- exercise_01_age_validation.py covers invalid, minor, and adult boundaries.
- exercise_02_process_users.py skips blank entries while preserving correct numbering.
- challenge_access_system.py is Project 2: validate role and active status repeatedly, allow quitting, and explain denial.

Trainer key points specific to this module

Have learners state rules in plain language before code. Draw branch paths and dry-run one loop iteration at a time. Test ages -1, 17, and 18. Watch indentation, ‘=‘ versus ‘==‘, conditions that can never run, and infinite while loops.

Portal connection and evidence

Validation controls registration; loops process database results. Learners should demonstrate accepted, rejected, invalid, and quit paths.

Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.
2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.